

Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: Ingeniería en Sistemas



FACULTAD DE INGENIERÍA

**UNIVERSIDAD DA VINCI
DE GUATEMALA**

Proyecto Final Estructuras de Arbóreas

Karen Fernanda Calderon Garcia

Carnet: 202406077

Estructura de Datos I

Objetivo del Proyecto

El presente documento documenta las características técnicas, eficiencia algorítmica y aplicaciones de las estructuras arbóreas más relevantes en la industria del software. Se incluye además la implementación práctica de un Árbol Binario de Búsqueda (BST) desarrollado desde cero en el lenguaje Java.

Fase 1: Fichas Técnicas

Árbol Binario de Búsqueda (BST)

Identificación: Estructura de datos jerárquica donde cada nodo tiene máximo dos hijos, manteniendo la propiedad de que los valores a la izquierda son menores y a la derecha mayores.

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(n)$
Inserción	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$

Características Mecánicas: Organiza datos de forma linealizada para búsquedas rápidas. No garantiza balanceo, lo que puede degenerar la estructura en una lista enlazada si los datos se insertan ordenados.

Ventaja Principal: Implementación simple y eficiente para conjuntos de datos dinámicos aleatorios.

Limitación Principal: Sensibilidad extrema al orden de inserción (falta de auto-balanceo).

Caso de Uso: Implementación básica de tablas de símbolos y diccionarios simples.

Árbol Balanceado (AVL)

Identificación: Árbol binario de búsqueda auto-balanceado donde la diferencia de alturas (factor de equilibrio) entre subárboles izquierdo y derecho es máximo 1.

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Características Mecánicas: Utiliza rotaciones (simples y dobles) tras cada inserción o eliminación para mantener el equilibrio estricto de altura.

Ventaja Principal: Búsquedas extremadamente rápidas debido a su balanceo perfecto.

Limitación Principal: El costo de reestructuración (rotaciones) es alto durante inserciones frecuentes.

Caso de Uso: Bases de datos in-memory donde las lecturas superan por mucho a las escrituras.

Árbol B / B+

Identificación: Árboles de búsqueda multi-vía balanceados, diseñados para trabajar eficientemente en sistemas de almacenamiento secundario (discos).

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log_b n)$	$O(\log_b n)$
Inserción	$O(\log_b n)$	$O(\log_b n)$
Eliminación	$O(\log_b n)$	$O(\log_b n)$

Características Mecánicas: Cada nodo puede contener múltiples claves y punteros. En el B+, todas las claves se repiten en las hojas, las cuales están vinculadas entre sí para recorridos secuenciales.

Ventaja Principal: Minimiza el número de accesos a disco gracias a su alta ramificación.

Limitación Principal: Complejidad elevada en la gestión de división y fusión de nodos.

Caso de Uso: Motores de bases de datos relacionales (ej. PostgreSQL, MySQL) y Sistemas de Archivos (NTFS).

Trie (Árbol de Prefijos)

Identificación: Árbol de búsqueda especializado en cadenas de texto, donde los nodos no almacenan la clave completa, sino fragmentos (caracteres).

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(L)$	$O(L)$
Inserción	$O(L)$	$O(L)$
Eliminación	$O(L)$	$O(L)$

Características Mecánicas: Cada ruta desde la raíz hacia abajo representa una palabra o prefijo común compartido por múltiples entradas ventaja.

Principal Búsqueda de prefijos extremadamente rápida, independiente de la cantidad de datos.

.Limitación Principal: Alto consumo de memoria por la cantidad de punteros nulos en cada nodo.

Caso de Uso: Diccionarios de Autocompletado y correctores ortográficos.

Implementación Práctica (BST en Java)

Lógica del Código

La implementación se realizó siguiendo el paradigma de Programación Orientada a Objetos, definiendo una clase `Nodo` para la unidad básica y una clase `Arbol` para la gestión de la estructura.

Inserción: Utiliza recursividad para navegar por el árbol comparando valores hasta encontrar una hoja nula.

Eliminación (El más complejo): Se contemplan los tres casos críticos:

Nodo Hoja: Se elimina simplemente referenciando a nulo.

Un Hijo: El hijo sube a ocupar el lugar del padre eliminado.

Dos Hijos: Se busca el sucesor in-order (el más pequeño del subárbol derecho), se reemplaza el valor y se elimina el sucesor de su posición original.

Análisis de Eficiencia La complejidad de mi método de búsqueda es en el caso promedio. Esto se debe a que, en cada paso de la búsqueda, descartamos aproximadamente la mitad del árbol restante, similar a una búsqueda binaria sin embargo, en el peor caso (si el árbol está desbalanceado), la complejidad puede ser.

Entrega de Repositorio en GitHub: <https://github.com/F13-C/ProyectoFinalArbol.git>